
CSC 412 – Operating Systems

Lab Session 08, Fall 2025

Tuesday, October 28th and Thursday, October 30th, 2025

What This Lab Is About

This lab is about named pipes. More specifically, it's about all the design decisions that you must make when planning for communications between processes.

Handout

For this lab, I give as a reminder code for a small FIFO-based client-server system under the form of two small initial programs `clientLab08.cpp` and `serverLab08.cpp`. The latter also initializes a small “database.”

Note: The “database” and the queries implemented here are hopelessly simple/naive/silly. The focus is not on the DB side of things but on the communication aspect. Of course, if we had more time, we could implement something that actually makes sense as a DB project.

What I really want you to get out of this lab is that it is your job to define the communication “language” between processes: What is being sent? In what format? Are replies expected? etc. At the same time, you have to take in to account: Will the server be dealing with more than one client? At the same time? Are there risks that clients’ messages can get mixed? Or skipped/ignored?

1 Task 1: The client send queries, the server prints out

The client-to-server pipe is already in place. The client sends to the server the zip code of a record, and the server prints out to the standard output the corresponding city name (and “record not found” if the zip is invalid).

Note that the handout does not allow for a “graceful” termination of the server. There is code in place to close the pipe when the server leaves the loop, but, since we are forced to kill the server to terminate, that code never executes, and so the pipe file has to be removed “by hand” until we fix the problem (in Task 3). Meanwhile, don’t forget to keep deleting the pipe between consecutive runs of your server.

2 Task 2: The server talks back

In this version, instead of printing out the result of the query, the server sends it back to the client via a named pipe.

The key problem here is: On what pipe does the server reply to the—*a priori* unrelated, unknown—client process? Since we are dealing with named pipes, how can the server know the name of the pipe on which to write back to the client, since the server didn't even know about the client when it got launched? How can we be sure that two clients won't end up reading on the same named pipe?

And of course, once you have made a design decision on the problem of naming the pipe the server writes on, you have to settle on a communication protocol for what it writes back. Again, this is your job!

3 Task 3: Different queries

In this version, the client can make three different kinds of queries when sending a zip code to the server:

- existence: is there a record for this zip code in the database?
- city name: what you did for Task 2;
- value: The other field in the database

While you are at it, now would be the time to solve the “server graceful exit” problem that we identified earlier. To keep things simple, let's say that we want to allow any client to request a shutdown of the server. So, that's one more thing that a client can send to the server, in addition to the queries listed above. So this makes up for a nice little communication protocol for you to define and implement.